

Block-Diffusion OD Planning — v4 Results (EOS-fixed backbones)

2026-07-27 · This report is built **entirely on the v4 round**: all seven mask backbones retrained after the EOS boundary fix, all discriminators and evaluations redone on top of them. The previous consolidated report (RESULTS_BD.pdf) remains the record of the pre-fix (v2) backbones and is referenced here only as the before-side of the fix. **Naming**: every v4 artifact carries the `_v4` tag — checkpoints `BD_porto_v3_normal_mask_blk{B}_v4_bd.pth` (the `v3` token is the *dataset* index `porto v3-0.05`, not a model round), discs `BDdisc_*_v4.pth`, pools `*_pool_v4_*`, eval tags `v4m* / v4f01m*`.

1. The EOS boundary fix

The training canvas is `[dst, v0, ..., vL-1=dst, END...END | PAD...]`, where `END` fills only the remainder of the block containing `dst` and later blocks are `PAD` (excluded from loss). Under that spec a sample whose `dst` lands **exactly on a block boundary** carries **zero END supervision** — a $1/B$ fraction of the data, and for $B=1$ every sample. The v2 blk1 backbone therefore never learned to terminate: unconditionally, 100% of its generations ran to the canvas cap (length 101, 50.5% invalid edges, valid rate 0.000). At $B\geq 2$ the same defect appeared as a systematic deficit of boundary-ending lengths: the generated share of lengths with $(L+1) \bmod B = 0$ was roughly half the real share (blk2 0.234 vs 0.500; blk32 0.011 vs 0.027).

Fix (`bd_models.py build_canvas()`): when the `END` tail would be empty, **extend the canvas by one full block of END tokens, included in the loss**. Every sample now trains ≥ 1 `END` target; for $B=1$ that is exactly one `END` token per sample. Verified by `smoke_test_bd.py` (canvas spec, both kernels' forward/backward and planning).

blk64 is unchanged by construction: no real path in this dataset has $L=63$ or 127 (boundary share ≈ 0.000), so the fix touches zero blk64 training samples; with identical seeding the v4 blk64 backbone reproduces v2's numbers exactly. It is retained for completeness.

2. Unconditional generation — before / after

20,000 open-ended generations per backbone (`plan(None, None)`), valid under 10% joint `ori+dst` condition dropout) — 140,000 paths in total across the seven blocks — scored on the `NORMAL` world. Real reference: length 23.2 ± 9.8 . **valid** = all edges legal; **invE** = invalid-edge rate; **KS** = length-distribution Kolmogorov–Smirnov distance vs real; **cap%** = generations hitting the canvas cap (failed to stop); **novel** = share not appearing verbatim in the real set; **bres** = boundary-residue share, generated/real — the length class starved of `END` supervision before the fix. v2 values in gray.

blk	valid v4 (v2)	invE v4 (v2)	len v4 (v2)	KS v4 (v2)	cap% v4 (v2)	novel v4 (v2)	bres gen/real v4 (v2 gen)
1	0.870 (0.000)	2.29% (50.47%)	25.2 (101.0)	0.045 (1.000)	0.9 (100.0)	0.396 (1.000)	—
2	0.737 (0.665)	3.30% (4.41%)	23.3 (24.1)	0.010 (0.042)	0.0 (0.1)	0.427 (0.519)	0.486/0.500 (0.234)
4	0.617 (0.517)	2.59% (3.78%)	23.8 (24.1)	0.026 (0.039)	0.0 (0.1)	0.662 (0.747)	0.235/0.250 (0.173)
8	0.325 (0.382)	5.88% (4.86%)	23.8 (24.5)	0.023 (0.055)	0.0 (0.0)	0.855 (0.834)	0.115/0.125 (0.102)
16	0.202 (0.184)	8.30% (9.09%)	24.4 (23.0)	0.056 (0.019)	0.0 (0.0)	0.902 (0.903)	0.054/0.061 (0.033)
32	0.141 (0.232)	9.37% (11.48%)	25.6 (16.7)	0.120 (0.278)	0.0 (0.0)	0.945 (0.935)	0.032/0.027 (0.011)
64	0.104 (0.104)	11.42% (11.42%)	24.1 (24.1)	0.056 (0.056)	0.0 (0.0)	0.971 (0.971)	0.000/0.000 (0.000)

- blk1 collapse fully resolved**: valid $0.000 \rightarrow 0.870$ (now the best unconditional generator), invalid edges $50.5\% \rightarrow 2.3\%$, length 101 (all capped) $\rightarrow 25.2$ with KS 0.045. AR needed exactly one trained `END` per sample.
- Boundary-residue deficit gone** at every block: generated shares now match real (blk2 0.486 vs 0.500; before 0.234).
- Validity–novelty trade-off persists**: small blocks are more valid but copy training routes more often (blk1 novel 0.396); blk4–8 balance validity 0.33–0.62 with novelty 0.66–0.86.
- Caveats**: blk8 ($0.382\rightarrow 0.325$) and blk32 ($0.232\rightarrow 0.141$) validity dipped — single-run training variance is not separable here; blk32's length calibration nevertheless improved sharply (KS $0.278\rightarrow 0.120$, mean $16.7\rightarrow 25.6$).

2a. Length distribution and edge-distribution match (JSEV)

Same 20,000 generations per block, scored against the **complete** real `NORMAL` set (1,930,710 shortest paths — the reference row). **KS** = Kolmogorov–Smirnov distance between the generated and real length distributions. **JSEV** is the project's standard edge-distribution metric (`utils/evaluate_new.py::Evaluator.calculate_divergences`): the Jensen–Shannon divergence between the real and generated directed-edge frequency distributions over the full $V \times V$ grid, in **nats** (scipy `rel ENTR`; divide by $\ln 2$ for bits). **R²** is computed on the same flattened edge distributions, and **Spearman ρ** is the rank correlation of edge counts — reported as *masked* (cells with a nonzero real count) / *all* cells. Lower JSEV and higher R^2/ρ are better.

blk	length mean $\pm$ std	median	p5 – p95	KS	JSEV (nats)	R ²	Spearman ρ (masked / all)
real (1,930,710)	23.21 $\pm$ 9.80	23	8 – 40	—	—	—	—
1	25.18 $\pm$ 12.51	24	9 – 43	0.045	0.0147	0.985	0.958 / 0.624
2	23.30 $\pm$ 9.66	23	8 – 40	0.010	0.0169	0.984	0.972 / 0.579

4	23.76 ± 9.77	23	9 – 40	0.026	0.0147	0.984	0.965 / 0.568
8	23.83 ± 9.85	23	9 – 41	0.023	0.0283	0.973	0.959 / 0.457
16	24.40 ± 9.78	24	9 – 41	0.056	0.0482	0.895	0.931 / 0.420
32	25.61 ± 10.15	26	8 – 42	0.120	0.0576	0.854	0.922 / 0.413
64	24.07 ± 8.94	24	10 – 39	0.056	0.0654	0.875	0.910 / 0.389

- **Length is well matched at every block:** means land within 2.4 of the real 23.21, medians within 3, and the 5–95% spread tracks the real 8–40 closely. blk2 is nearly exact (23.30 ± 9.66, KS 0.010); blk1 has the heaviest tail (std 12.51) and blk32 the worst length calibration (KS 0.120). For scale, the pre-fix blk1 backbone produced 101 ± 0.00 (every sample capped) — see §2.
- **Edge-distribution fidelity degrades monotonically with block size:** JSEV 0.0147 (blk1, blk4) → 0.0654 (blk64), R² 0.985 → 0.875. Small blocks reproduce *which roads get used how often* much more faithfully.
- **The two Spearman columns separate two failure modes:** on edges that really occur, the usage ranking is reproduced very well at every block (p 0.91–0.97); across all V² cells the correlation is far lower (0.39–0.62) and falls with block size, because larger blocks spend probability mass on edges the real data never uses — the same illegal-edge rate (2.3% → 11.4%) reported in §2.
- RMSE / MAE on the edge distributions follow the same ordering (blk1 2.83e-06 / 8.48e-08 → blk64 8.23e-06 / 2.38e-07) and are recorded in `sets_res/uncond_jsev_v4.json` .

2b. Unconditional EM/PC — each sample's own endpoints as its OD

EM/PC need *no given* OD: `evaluate_em_pc` takes the generated path's own (start, end) as its OD and looks it up in the reference shortest-path data. Since training paths are shortest paths, **uncond EM = "is the sample a shortest path between its own endpoints"** — a distribution-fidelity metric. OD coverage (share of generated ODs present in the reference) is ≈1.0 at every block, so the measurement is clean. 3,000 samples/block, seed 7. **novel∧EM** is the synthetic-data currency: a *new* shortest path for a known OD, i.e. an alternative equal-length route absent from the dataset.

blk	valid	EM	PC	novel	valid∧novel	novel∧EM	OD cov
1	0.864	0.640	0.716	0.406	0.271	0.046	1.000
2	0.745	0.629	0.675	0.425	0.170	0.054	1.000
4	0.633	0.401	0.481	0.652	0.285	0.053	0.999
8	0.346	0.178	0.230	0.851	0.197	0.029	0.998
16	0.197	0.113	0.141	0.907	0.104	0.019	0.997
32	0.130	0.058	0.080	0.951	0.081	0.009	0.998
64	0.105	0.035	0.056	0.973	0.078	0.008	0.995

- **blk1/2 dominate shortest-path fidelity** (EM 0.63–0.64 vs blk4 0.40); on the novelty-quality frontier blk2/4 hold only a slim edge (novel∧EM 0.053–0.054 vs blk1's 0.046; valid∧novel blk4 0.285 vs blk1 0.271).
- **Novelty caveat:** this dataset is shortest paths, so per OD the answer is near-unique — a *perfect* generative model would show *low* novelty. blk1's verbatim copies are partly a sign of learning the distribution, not only memorization; the novelty axis is intrinsically tilted toward larger blocks, and even so their edge is ≤1%p absolute.

2c. Diversity among VALID generations — the intuition test

Raw novelty rises with block size (0.406 → 0.973, within-block mean-field sampling adds combinatorial variety) — but diversity of invalid paths is noise. Restricting to the valid subset and benchmarking against an equal-size real sample (metrics: distinct-OD share, distinct directed edges used, edge-usage Shannon entropy, mean pairwise Jaccard similarity of edge sets — higher = more mutually similar). **n valid** is the size of that subset: how many of the 20,000 generations have every edge legal, i.e. the sample count each row is computed on.

	n valid	distinct OD	edge support	edge entropy	pairwise Jaccard
real (20k)	20,000	1.000	3,856	10.62 b	0.0124
blk1	17,395	0.995	3,793	10.54 b	0.0132
blk2	14,744	0.996	3,728	10.47 b	0.0151
blk4	12,338	0.995	3,753	10.49 b	0.0131
blk8	6,501	0.996	3,541	10.30 b	0.0149
blk16	4,046	0.995	3,273	10.09 b	0.0189
blk32	2,829	0.995	3,124	9.99 b	0.0186
blk64	2,080	0.998	2,889	10.03 b	0.0186

- **Conditioned on validity, diversity decreases with block size** — the opposite of the raw-novelty picture: edge support 3,793 → 2,889, entropy 10.54 → 10.03 b, pairwise similarity 0.0132 → 0.0186. Selection effect: at large B only 10–14% of samples are valid, and the survivors concentrate on short, common corridors.
- **blk1's valid set matches the real distribution's diversity almost exactly** (98% of the real edge support, entropy –0.08 b, near-identical pairwise similarity). The apparent diversity of large blocks is mostly the diversity of invalid noise.

- **Net reading for the block-size choice:** quality, fidelity and *valid* diversity all favor small blocks; the only quantitative diffusion edge left in open-ended generation is the slim novel $\wedge$ EM margin of blk2–4. (Sampling-speed differences measured earlier are implementation overhead, not algorithmic — first-hitting NFE is block-size-independent; see §8.)

3. Conditional planning — held-out real OD pairs (normal world)

1,000 held-out real shortest-path pairs, 3 generation seeds, mean $\pm$ std (`eval_bd_sweep_controlled --suffix v4`). **arr** = reaches destination; **v $\wedge$ h** = valid $\wedge$ hit; EM/PC per §1.2 of the v3 report. All RAW (no refine).

blk	arrival	valid	valid $\wedge$ hit	EM	PC	s/path
1	0.994 $\pm$ 0.002	0.952$\pm$0.005	0.948$\pm$0.005	0.843$\pm$0.007	0.887$\pm$0.006	0.082
2	0.948 $\pm$ 0.005	0.818 $\pm$ 0.006	0.789 $\pm$ 0.008	0.756 $\pm$ 0.005	0.782 $\pm$ 0.006	0.060
4	0.937 $\pm$ 0.004	0.878 $\pm$ 0.008	0.826 $\pm$ 0.005	0.786 $\pm$ 0.010	0.823 $\pm$ 0.010	0.035
8	0.932 $\pm$ 0.012	0.728 $\pm$ 0.003	0.687 $\pm$ 0.010	0.671 $\pm$ 0.004	0.695 $\pm$ 0.003	0.030
16	0.926 $\pm$ 0.000	0.676 $\pm$ 0.013	0.641 $\pm$ 0.008	0.600 $\pm$ 0.010	0.631 $\pm$ 0.011	0.031
32	0.879 $\pm$ 0.004	0.585 $\pm$ 0.010	0.529 $\pm$ 0.011	0.499 $\pm$ 0.006	0.533 $\pm$ 0.008	0.032
64	0.839 $\pm$ 0.005	0.430 $\pm$ 0.014	0.373 $\pm$ 0.010	0.366 $\pm$ 0.012	0.391 $\pm$ 0.012	0.033

Conditional quality still favors small blocks (dst-anchored first-hit termination is untouched by the EOS fix). Together with §2, the division of roles is now clean: **conditional planning** $\rightarrow$ **small blocks; unconditional / open-ended synthesis** $\rightarrow$ **diffusion blocks only** (blk1 was unusable there before the fix and is competitive after it, but larger blocks retain the novelty edge and guidance headroom).

4. Guidance on the exceptional scenario — fam 0.05 (except_0, RAW)

Reserved 1,000 except_0 pairs (never seen by any disc), seed 7, first-hitting, per-block model-negative discriminators (`_v4`), n_is=100. Columns per method: valid / arrival / **valid $\wedge$ arrival** / EM / PC, all RAW. Plain IW is omitted from the tables (base < IW < adj+IW ordering holds at every block; e.g. blk4 valid 0.279/0.332/0.389). **Seen** = disc trained on except_0 (1%, excluding the reserved rows); **Unseen** = disc trained on except 1–99, applied zero-shot.

4.1 Seen

blk	base valid	base arr	base v $\wedge$ a	base EM	base PC	adj+IW valid	adj+IW arr	adj+IW v $\wedge$ a	adj+IW EM	adj+IW PC
1	0.302	0.994	0.300	0.271	0.283	0.928	0.567	0.540	0.374	0.425
2	0.251	0.949	0.238	0.233	0.240	0.492	0.846	0.401	0.388	0.423
4	0.279	0.932	0.262	0.243	0.258	0.389	0.873	0.328	0.334	0.357
8	0.229	0.938	0.214	0.202	0.214	0.331	0.862	0.273	0.295	0.309
16	0.206	0.923	0.194	0.180	0.190	0.298	0.865	0.250	0.230	0.255
32	0.185	0.870	0.160	0.157	0.168	0.265	0.825	0.215	0.219	0.237
64	0.141	0.827	0.119	0.129	0.134	0.229	0.795	0.162	0.176	0.198

4.1b IW only, no adjacency masking — seen

Same discriminator as §4.1 with Lemma-3 proposal masking OFF, so this isolates what the discriminator reweighting contributes on its own. Same columns, all RAW.

blk	base valid	base arr	base v $\wedge$ a	base EM	base PC	IW valid	IW arr	IW v $\wedge$ a	IW EM	IW PC
1	0.302	0.994	0.300	0.271	0.283	0.324	0.991	0.321	0.283	0.298
2	0.251	0.949	0.238	0.233	0.240	0.328	0.951	0.312	0.306	0.315
4	0.279	0.932	0.262	0.243	0.258	0.332	0.936	0.304	0.295	0.310
8	0.229	0.938	0.214	0.202	0.214	0.294	0.923	0.254	0.270	0.279
16	0.206	0.923	0.194	0.180	0.190	0.260	0.917	0.236	0.211	0.230
32	0.185	0.870	0.160	0.157	0.168	0.244	0.867	0.207	0.204	0.220
64	0.141	0.827	0.119	0.129	0.134	0.201	0.854	0.157	0.160	0.177

4.2 Unseen (disc = except 1–99 $\rightarrow$ except_0; base identical to §4.1)

blk	base valid	base arr	base v $\wedge$ a	base EM	base PC	adj+IW valid	adj+IW arr	adj+IW v $\wedge$ a	adj+IW EM	adj+IW PC
-----	------------	----------	-------------------	---------	---------	--------------	------------	---------------------	-----------	-----------

1	0.302	0.994	0.300	0.271	0.283	0.925	0.558	0.530	0.362	0.412
2	0.251	0.949	0.238	0.233	0.240	0.488	0.848	0.415	0.367	0.408
4	0.279	0.932	0.262	0.243	0.258	0.390	0.875	0.331	0.335	0.357
8	0.229	0.938	0.214	0.202	0.214	0.331	0.863	0.273	0.295	0.308
16	0.206	0.923	0.194	0.180	0.190	0.300	0.868	0.254	0.237	0.261
32	0.185	0.870	0.160	0.157	0.168	0.268	0.828	0.219	0.222	0.240
64	0.141	0.827	0.119	0.129	0.134	0.224	0.798	0.162	0.178	0.197

4.2b IW only, no adjacency masking — unseen

As §4.1b, with the zero-shot (except 1–99) discriminator.

blk	base valid	base arr	base v _Λ a	base EM	base PC	IW valid	IW arr	IW v _Λ a	IW EM	IW PC
1	0.302	0.994	0.300	0.271	0.283	0.321	0.992	0.319	0.281	0.296
2	0.251	0.949	0.238	0.233	0.240	0.330	0.951	0.316	0.305	0.316
4	0.279	0.932	0.262	0.243	0.258	0.331	0.935	0.302	0.295	0.310
8	0.229	0.938	0.214	0.202	0.214	0.298	0.923	0.258	0.275	0.284
16	0.206	0.923	0.194	0.180	0.190	0.264	0.919	0.242	0.215	0.234
32	0.185	0.870	0.160	0.157	0.168	0.246	0.869	0.209	0.208	0.223
64	0.141	0.827	0.119	0.129	0.134	0.199	0.857	0.157	0.161	0.177

- **The v2 conclusions replicate on the fixed backbones:** adj+IW improves the strict raw success (valid_Λarrival) at every block, trading arrival for a larger validity gain; blk1 is the extreme case (valid 0.302→0.928, arrival 0.994→0.567, v_Λa 0.300→0.540).
- **Seen/unseen gap ≤0.015 everywhere** (e.g. v_Λa blk1 0.540/0.530, blk4 0.328/0.331) — zero-shot scenario transfer confirmed again; the discriminator-level analysis is in [DISC_VALIDITY.pdf](#).
- adj+IW ESS ≈ 95.5 (blk64) – 99.7 (blk1): the disc tilt stays gentle per reveal; the blk1 jump is driven by Lemma-3 adjacency masking, not the disc.

5. Controlled comparison with the published baseline: D-CBG (Schiff et al.)

Plug-in port of *Simple Guidance Mechanisms for Discrete Diffusion Models* (`dcbg_plugin.py` , their `_cbg_denoise` logic verbatim), re-run on the v4 backbones. **What differs from IW is method-intrinsic only:** D-CBG scores *noisy* states $p_{\Phi}(y \mid x_t, t)$ and tilts the per-position categorical, whereas IW scores *clean* partial paths and reweights joint candidates. Everything else is matched — same adjacency-aware architecture (`-adj 1`), same 1% data budget and split seeds, same model-generated negatives (each block's own v4 unconditional pool, corrupted by the model's forward process), same 1,000 reserved `except_0` pairs, seed 7.

γ = 1 for both methods. Earlier rounds ran D-CBG at γ = 4 while IW ran at its default γ = 1; that compared the two at different guidance strengths. Here both use the exact, untempered form — IW's $w_n = \exp(\gamma \cdot \ell_{\Phi})$ reduces to the plain density ratio $D/(1-D)$, and D-CBG's $\log p_{\Phi} + \gamma \cdot \log p_{\Phi}$ reduces to the plain Bayes tilt. (For reference, the v1-era γ sweep on v2 backbones: D-CBG blk4 valid 0.262 / 0.273 / 0.289 at γ = 1 / 2 / 4 — the method needs γ ≫ 1 to move at all.)

Exact vs first-order is a runtime flag on the same classifier (`-approx`), not a separate model. **exact:** enumerate all V = 1,393 tokens at the reveal position — under first-hitting only that one position is consumed per step, so this is exact, not an approximation. **first-order:** their Taylor variant, one classifier forward+backward on the one-hot input per reveal.

5.1 Seen (classifier trained on except_0) — same columns as §4.1, all RAW

blk	base valid	base arr	base v _Λ a	base EM	base PC	D-CBG valid	D-CBG arr	D-CBG v _Λ a	D-CBG EM	D-CBG PC
1	0.302	0.994	0.300	0.271	0.283	0.324	0.995	0.321	0.283	0.298
2	0.251	0.949	0.238	0.233	0.240	0.278	0.946	0.264	0.255	0.264
4	0.279	0.932	0.262	0.243	0.258	0.298	0.939	0.281	0.258	0.275
8	0.229	0.938	0.214	0.202	0.214	0.239	0.927	0.219	0.208	0.222
16	0.206	0.923	0.194	0.180	0.190	0.226	0.917	0.207	0.197	0.208
32	0.185	0.870	0.160	0.157	0.168	0.206	0.872	0.174	0.170	0.184
64	0.141	0.827	0.119	0.129	0.134	0.156	0.813	0.126	0.132	0.142

5.2 Unseen (classifier trained on except 1–99 → except_0 zero-shot; base identical to §5.1)

blk	base valid	base arr	base v _Λ a	base EM	base PC	D-CBG valid	D-CBG arr	D-CBG v _Λ a	D-CBG EM	D-CBG PC
-----	------------	----------	-----------------------	---------	---------	-------------	-----------	------------------------	----------	----------

1	0.302	0.994	0.300	0.271	0.283	0.322	0.994	0.319	0.279	0.295
2	0.251	0.949	0.238	0.233	0.240	0.280	0.948	0.266	0.255	0.265
4	0.279	0.932	0.262	0.243	0.258	0.299	0.937	0.280	0.256	0.274
8	0.229	0.938	0.214	0.202	0.214	0.243	0.931	0.222	0.211	0.225
16	0.206	0.923	0.194	0.180	0.190	0.225	0.920	0.207	0.194	0.205
32	0.185	0.870	0.160	0.157	0.168	0.196	0.869	0.166	0.162	0.175
64	0.141	0.827	0.119	0.129	0.134	0.151	0.820	0.125	0.130	0.139

5.3 First-order (Taylor) variant, seen — same columns as §5.1

Same classifiers as §5.1/§5.2 with `-approx 1` : one classifier forward+backward on the one-hot input per reveal, instead of enumerating all V = 1,393 tokens. All RAW.

blk	base valid	base arr	base v _{AA}	base EM	base PC	D-CBG (1st-order) valid	D-CBG (1st-order) arr	D-CBG (1st-order) v _{AA}	D-CBG (1st-order) EM	D-CBG (1st-order) PC
1	0.302	0.994	0.300	0.271	0.283	0.301	0.994	0.299	0.271	0.283
2	0.251	0.949	0.238	0.233	0.240	0.251	0.948	0.237	0.235	0.242
4	0.279	0.932	0.262	0.243	0.258	0.281	0.930	0.262	0.246	0.261
8	0.229	0.938	0.214	0.202	0.214	0.228	0.936	0.213	0.202	0.214
16	0.206	0.923	0.194	0.180	0.190	0.206	0.922	0.194	0.181	0.191
32	0.185	0.870	0.160	0.157	0.168	0.184	0.871	0.159	0.155	0.166
64	0.141	0.827	0.119	0.129	0.134	0.142	0.825	0.120	0.130	0.135

5.4 First-order (Taylor) variant, unseen — same columns as §5.2

blk	base valid	base arr	base v _{AA}	base EM	base PC	D-CBG (1st-order) valid	D-CBG (1st-order) arr	D-CBG (1st-order) v _{AA}	D-CBG (1st-order) EM	D-CBG (1st-order) PC
1	0.302	0.994	0.300	0.271	0.283	0.302	0.994	0.300	0.271	0.283
2	0.251	0.949	0.238	0.233	0.240	0.252	0.948	0.239	0.234	0.241
4	0.279	0.932	0.262	0.243	0.258	0.282	0.930	0.265	0.246	0.261
8	0.229	0.938	0.214	0.202	0.214	0.230	0.934	0.214	0.203	0.215
16	0.206	0.923	0.194	0.180	0.190	0.207	0.923	0.195	0.182	0.192
32	0.185	0.870	0.160	0.157	0.168	0.183	0.870	0.158	0.155	0.166
64	0.141	0.827	0.119	0.129	0.134	0.141	0.824	0.119	0.129	0.134

Inert at every block, in both regimes: validity stays within ±0.003 of base (blk1 0.301 vs 0.302; blk4 0.281 vs 0.279; blk64 0.142 vs 0.141), and v_{AA} / EM / PC likewise. Contrast the exact variant of §5.1 (blk4 0.298, blk64 0.156). The same conclusion the v3 report reached on the v2 backbones — the gradient at the one-hot input does not approximate the effect of actually substituting a token. Cost: ≈0.013–0.019 s/path (§5.6), the cheapest guided option and the only one that buys nothing.

5.5 All five methods side by side at γ = 1 (raw validity)

The same numbers as §4.1 and §5.1 collected into one table — this is the only place where base, both D-CBG variants and both IW variants appear together at the same guidance strength.

blk	base	D-CBG first-order	D-CBG exact	IW	adj+IW
1	0.302	0.301	0.324	0.324	0.928
2	0.251	0.251	0.278	0.328	0.492
4	0.279	0.281	0.298	0.332	0.389
8	0.229	0.228	0.239	0.294	0.331
16	0.206	0.206	0.226	0.260	0.298
32	0.185	0.184	0.206	0.244	0.265
64	0.141	0.142	0.156	0.201	0.229

- At equal guidance strength IW dominates D-CBG at every block ≥ 2: over base, exact D-CBG lifts validity by +1.0 to +2.7%p, IW by +2.2 to +7.7%p, adj+IW by more. The ordering base < D-CBG(fo) ≈ base < D-CBG(exact) < IW < adj+IW holds at every block.

- **blk1 is a tie (0.324 both)**, and for a measurable reason: comparing generated paths one by one, D-CBG changes only 5% of them relative to base (950/1000 identical) and IW 18% (824/1000). At B = 1 there is no within-block candidate diversity to exploit (ESS ≈ 99.7), so no reweighting scheme can do much — the blk1 gains in §4 come from Lemma-3 adjacency masking, not from any classifier.
- **The first-order variant is inert on v4 too** (§5.3–§5.4): valid within ±0.003 of base at every block, in both regimes — the same conclusion the v3 report reached on v2 backbones.
- **D-CBG also transfers zero-shot** (seen/unseen gap ≤0.010 on every metric), so the generalization finding of §7 is not specific to the IW discriminator — it follows from feeding the scenario adjacency as an input.
- **Removed-edge usage stays at base level** (~4.9–5.1% vs base ~5%), i.e. the per-position tilt barely encodes the scenario constraint; IW's Lemma-3 masking is what drives that number to ~3%.

5.6 Wall-clock and scorer calls — like-for-like

What the timer measures, exactly: the wall-clock of *one planner call on a batch of 100 OD pairs* — from handing in the (origin, destination) lists to receiving the finished vertex paths — divided by 100. **Inside:** denoiser forward passes, guidance scoring (discriminator / classifier), candidate sampling, per-reveal bookkeeping, the planner's own CPU sync and stop scan, path assembly. **Outside:** checkpoint/data loading, P1/P3 post-processing, EM/PC/DTW scoring, and everything else after the planner returns. So the number is the **amortised per-path cost when 100 paths are planned concurrently**, *not* the latency of planning one path on its own (which is far higher per path on an under-utilised GPU).

Fairness controls (`time_guidance.py`): identical OD pairs, identical batch, identical seed for every method; one method at a time, sequentially, on **one GPU that the script verifies is idle** — it queries `nvidia-smi` for resident compute apps and refuses to run if any are present; a **warm-up batch per (block, method) that is executed and discarded**, so CUDA context creation and kernel autotuning never land in a reported number; every timed region wrapped in `torch.cuda.synchronize()` ; and **3 timed batches per cell that are different slices of the reserved pool** (pairs 1–100, 101–200, 201–300), identical across methods. Reported as mean ± std over those 3 batches, so the spread reflects batch composition as well as run-to-run jitter. 300 timed paths per cell, 35 cells.

Not comparable to: (i) the 0.63 s/path printed by the evaluation run for D-CBG-exact at blk4 — that is `eval_dcbg.py` 's `sec_per_path` , whose timer spans generation *plus* P1/P3 splicing and EM/PC scoring, with four processes competing for CPU, against 0.328 here for planning alone; (ii) the s/path column of §3, from `eval_bd_sweep_controlled.py` , which spans generation plus EM/PC, DTW and length scoring over 1,000 paths. **Scorer calls** = discriminator/classifier sequence evaluations per path, the implementation-independent cost (L ≈ 22 revealed tokens on a real path): IW = L·n_{is}, D-CBG exact = L·V, first-order = L forward+backward passes.

blk	base	IW	adj+IW	D-CBG first-order	D-CBG exact
1	0.0050 ± 0.0002	0.0264 ± 0.0028	0.0934 ± 0.0000	0.0132 ± 0.0005	0.3126 ± 0.0242
2	0.0044 ± 0.0003	0.0262 ± 0.0026	0.0531 ± 0.0285	0.0127 ± 0.0007	0.3052 ± 0.0323
4	0.0043 ± 0.0002	0.0277 ± 0.0018	0.0278 ± 0.0018	0.0130 ± 0.0006	0.3281 ± 0.0222
8	0.0045 ± 0.0000	0.0315 ± 0.0000	0.0316 ± 0.0000	0.0135 ± 0.0001	0.3768 ± 0.0000
16	0.0056 ± 0.0000	0.0435 ± 0.0000	0.0437 ± 0.0000	0.0156 ± 0.0000	0.5313 ± 0.0001
32	0.0060 ± 0.0000	0.0517 ± 0.0000	0.0520 ± 0.0000	0.0158 ± 0.0000	0.6385 ± 0.0001
64	0.0072 ± 0.0000	0.0669 ± 0.0000	0.0676 ± 0.0000	0.0163 ± 0.0000	0.8360 ± 0.0002
scorer calls / path	0	≈2,200	≈2,200	≈22 (fwd+bwd)	≈30,600

s/path, mean ± std over 3 distinct batches of 100. Reveal steps per batch (blk1 / blk4 / blk64): IW 48 / 50 / 62, adj+IW 100 / 50 / 62.

- **Exact D-CBG costs 12× IW** for a smaller gain (blk4: 0.328 vs 0.028 s/path; blk64: 0.836 vs 0.067). The gap is structural — per reveal it scores all V = 1,393 substitutions, while IW scores n_{is} = 100 sampled candidates.
- **The first-order variant is the cheapest guided option (0.013–0.016 s/path) and buys nothing** — cheaper than IW, but statistically indistinguishable from unguided base (§5.3–§5.4).
- **Base planning is nearly flat in block size** (0.0043–0.0072 s/path) — first-hitting spends one forward per revealed token regardless of B, and only the growing canvas adds cost. **Both guided families scale mildly** for the same reason. **The exception is adj+IW at blk1/2** (0.093 / 0.053 s/path): adjacency masking makes many paths overshoot the destination, so the sampler runs more reveals per batch (100 vs 48 at blk1) — the same effect that shows up as arrival 0.567 in §4.1. blk2's large std (±0.0285) is that same instability: the three batches did not all hit the reveal cap.
- **The ±0.0000 std at blk8 and above is genuine, not a transcription artefact.** Verified by printing raw per-batch wall-clock: blk8 IW = 3.134 / 3.138 / 3.138 s, blk16 D-CBG-exact = 53.246 / 53.247 / 53.244 s. At batch 100 the number of reveal steps is fixed regardless of which pairs the batch contains — the planner runs until the *whole batch* stops, and with 100 paths the longest one is always near the pool maximum — so the wall-clock is deterministic to <0.1%. At blk1–4 the reveal count still varies between batches, which is where the nonzero std comes from.
- **Caveat on the first-order column:** it varies ±20% *between processes* (blk8: 0.0135 in the main run vs 0.0166 in a re-run; blk16: 0.0156 vs 0.0188), because that cell is bound by kernel-launch overhead rather than by compute, so it is sensitive to whatever else the driver is doing. Read it as ≈0.013–0.019, not to four digits.
- **Caveat:** wall-clock reflects this implementation (no KV caching, no parallel within-block commits) and this GPU; the scorer-call row is the portable comparison.

5.6.1 Does batch = 100 disadvantage a particular method?

IW runs n_{is} = 100 candidates per path internally, so a batch of 100 already means 10⁴ sequences scored per reveal — the concern is whether that ceiling penalises it relative to methods that keep amortising. Same protocol, blk4, batch swept 25 → 400 (single timed batch per cell here, so no std):

batch	base	IW	adj+IW	D-CBG first-order	D-CBG exact
-------	------	----	--------	-------------------	-------------

25	0.0140	0.0333	0.0338	0.0542	0.2653
50	0.0077	0.0284	0.0287	0.0292	0.2889
100	0.0044	0.0277	0.0279	0.0157	0.3285
200	0.0032	0.0263	0.0276	0.0084	0.3274
400	0.0030	0.0304	0.0285	0.0055	0.3599

- **IW is flat in batch size** (0.0333 → 0.0304 across 25 → 400, no monotone trend) — it is already compute-bound at $n_{\text{is}} = 100$, so batch 100 costs it nothing. Same for adj+IW (0.0338 → 0.0285) and D-CBG exact (0.265 → 0.360, in fact mildly worse at large batch).
- **The method batch = 100 does penalise is D-CBG first-order**, which keeps amortising all the way out: 0.0542 at bs 25 → 0.0055 at bs 400, i.e. **2.9× faster at bs 400 than at the bs 100 used in the main table**. The main table therefore understates first-order's speed at large batch — which changes no conclusion, since its problem is that it changes nothing (§5.3), not that it is slow.
- Base also keeps amortising (0.0140 → 0.0030). No method ran out of memory at bs 400 (blk4), so the batch was not capped by IW's memory footprint.

6. Guidance — fam 0.1 (except_0, RAW; same protocol)

Same pipeline at edge-remove ratio 0.1: a much harder scoring world (twice the removed edges) — base validity drops to 0.04–0.12. Per-block `_v4` discs trained on fam 0.1 scenarios; unseen = except 1–99 → except_0 zero-shot.

6.1 Seen

blk	base valid	base arr	base v $\wedge$ a	base EM	base PC	adj+IW valid	adj+IW arr	adj+IW v $\wedge$ a	adj+IW EM	adj+IW PC
1	0.115	0.993	0.114	0.108	0.110	0.927	0.250	0.234	0.144	0.177
2	0.096	0.960	0.092	0.089	0.092	0.245	0.717	0.163	0.174	0.200
4	0.089	0.949	0.084	0.075	0.081	0.169	0.841	0.137	0.141	0.152
8	0.077	0.933	0.071	0.073	0.074	0.143	0.804	0.101	0.123	0.131
16	0.075	0.934	0.069	0.067	0.071	0.132	0.819	0.091	0.103	0.115
32	0.072	0.884	0.061	0.062	0.066	0.118	0.793	0.076	0.088	0.099
64	0.040	0.865	0.032	0.034	0.036	0.119	0.744	0.068	0.094	0.105

6.2 Unseen (base identical to §6.1)

blk	base valid	base arr	base v $\wedge$ a	base EM	base PC	adj+IW valid	adj+IW arr	adj+IW v $\wedge$ a	adj+IW EM	adj+IW PC
1	0.115	0.993	0.114	0.108	0.110	0.932	0.248	0.232	0.145	0.177
2	0.096	0.960	0.092	0.089	0.092	0.243	0.720	0.164	0.174	0.198
4	0.089	0.949	0.084	0.075	0.081	0.168	0.841	0.136	0.141	0.151
8	0.077	0.933	0.071	0.073	0.074	0.145	0.803	0.101	0.124	0.132
16	0.075	0.934	0.069	0.067	0.071	0.134	0.819	0.092	0.102	0.115
32	0.072	0.884	0.061	0.062	0.066	0.117	0.800	0.079	0.084	0.097
64	0.040	0.865	0.032	0.034	0.036	0.121	0.745	0.070	0.095	0.105

- **Guidance survives the harder world**: relative gains grow (blk64 valid 0.040→0.119, ~3×; v $\wedge$ a 0.032→0.068–0.070), though absolute levels are low — at 10% edge removal the base model's prior is badly mismatched with the scenario graph.
- **The arrival cost of hard masking grows with difficulty**: blk1 adj+IW arrival collapses to 0.25 (legal-only reveals overrun the destination far more often when twice the edges are gone). v $\wedge$ a still doubles over base (0.114→0.234). Endpoint post-processing (P3) is the natural complement, as in the v3 report.
- **Zero-shot transfer holds at fam 0.1 too**: seen/unseen gap ≤ 0.003 in v $\wedge$ a across all blocks.
- adj+IW ESS 97.2 (blk64) – 99.6 (blk1); orderings base < IW < adj+IW hold at every block.

7. Discriminator validity (summary)

Full analysis in `DISC_VALIDITY.pdf` / `DISC_VALIDITY_K0.pdf` . Headlines: (i) seen vs unseen discs score held-out inputs near-identically (logit Pearson 0.81–0.98); (ii) on the model-negative task, zero-shot AUC loss is ≤ 0.05 in both probe directions — most of the signal (edge legality vs the input adjacency, model artifacts) is scenario-agnostic by construction; (iii) the harder data-negative probe (real vs real) separates the two training regimes: the single-scenario e0 disc drops –0.100 (fam 0.05) / –0.084 (fam 0.1) AUC zero-shot, while the 99-scenario e99 disc is nearly flat — multi-scenario training is what buys genuine scenario-specific generalization; (iv) the v2 blk1 "best AUC" was an artifact of its degenerate negative pool and disappears after the EOS fix (0.985 → 0.870), confirming the negative-pool confound analysis.

8. Notes & provenance

- **Every number in this report is RAW** (no P1/P3, no refine — `use_refine=False` in the conditional sweep). The post-processed counterparts of §4, §5 and §6 are in a separate file, `RESULTS_BD_V4_POSTPROC.pdf`.
- D-CBG round (§5): `run_v4_dcbg_train.sh` (28 classifiers, model-negatives only) then `run_v4_dcbg_eval.sh` (28 evals, exact + first-order, $\gamma=1$) and `time_guidance.py`; artifacts `sets_res/dcbg_table_v4_f005.json`, `guidance_timing_v4.json`.
- Pipeline: `run_v4_all.sh`, single tmux chain, 2026-07-27 00:48–04:22 KST (Stage A retrain $\approx$ 1h40m; 28 guided evals $\approx$ 1h). Raw artifacts: `sets_res/va_table_v4_f005.json`, `va_table_v4_f01.json`, `uncond_pool_eval_v4.json`, `disc_gen_eval_f{005,01}{,_data}_v4.json`, logs `sets_log/v4_*.log`.
- Training identical to v2 except the canvas fix: `train_porto_bd.sh mask B gpu v4` (1 epoch, bs 32, lr 5e-4, drop_cond 0.1 — ori and dst are always dropped *jointly*, never individually).
- The conditional-eval script's output prefix is hard-coded `V2_`; its per-path CSVs under that prefix now hold v4 results (v2 aggregates preserved in `sets_log/v2_ctrl_mask.log` and the v3 report).
- **s/path caveat:** with first-hitting the algorithmic NFE is block-size-independent (one forward per reveal, total $\approx$ generated length). The measured blk1 slowdown (0.082 vs $\sim$ 0.033 s/path) is per-block implementation overhead — a CPU sync + Python stop-scan + attention-mask rebuild per block, paid per *token* at B=1 — plus batch-straggler synchronization; not an algorithmic advantage of larger blocks (no KV caching / parallel within-block commits are implemented).
- Old (pre-fix) checkpoints/discs/pools were left in place; nothing v4 overwrites them (distinct names). They can be deleted once this report is accepted.